EduAssist AI: LLMOps Benchmarking & Prompt Lifecycle Report

Course: Machine Learning Operations (AIMLCZG523)
Student: Vishal Anand | **BITS ID:** 2025AE05692
Email: 2025AE05692@wilp.bits-pilani.ac.in
Assignment: Experiential Learning Assignment - 2
Tracking Store: sqlite:///mlflow.db (MLflow 3.18)
Serving Backend: Local Ollama (llama3.2:1b)

1. Executive Summary & Objective

Large Language Model Operations (LLMOps) extends classical MLOps paradigms by managing the lifecycle of applications driven by foundational language models. Classical MLOps focuses on tabular dataset versioning, feature engineering, and neural weight training pipelines. In contrast, LLMOps governs prompt templates, context retrieval (RAG), guardrails, and non-deterministic generative text as versioned production software assets. Operational excellence in LLMOps centers around controlling API token costs, minimizing autoregressive latency, and enforcing deterministic output contracts.

In this experiential assignment, we operationalize **EduAssist AI**, an automated academic question-answering assistant for undergraduate computer science students. Using **MLflow 3.18** and a local inference backend powered by **Ollama (llama3.2:1b)**, we evaluate three prompt design paradigms across latency, token economics, schema compliance, and lifecycle promotion.

Benchmark Dimension	Prompt v1 (Basic)	Prompt v2 (Structured)	Prompt v3 (JSON Schema)
Prompt Architecture	Unconstrained baseline	Persona & concise phrasing	Strict JSON schema contract
Mean Response Latency	3.463s	1.999s (-42.3%)	**0.651s (-81.2%)**
Avg Completion Tokens	448.6 tokens	363.2 tokens	**112.2 tokens (-75.0%)**
Valid JSON Pass Rate	0.0% (0 / 5 valid)	0.0% (0 / 5 valid)	**100.0% (5 / 5 valid)**
Marginal API Token Cost	$0.00 (Self-hosted local)	$0.00 (Self-hosted local)	$0.00 (Self-hosted local)
Projected Cloud Cost / 10k Req	$2.76 / month	$2.28 / month	**$0.82 / month (-70.3%)**
MLflow Registry Promotion	Version 1	Version 2	**Version 3 (@production)**

2. Experimental Setup & Question Corpus

All evaluations executed against a standardized benchmark corpus of 5 foundational computer science questions: **Q1:** Difference between supervised and unsupervised learning; **Q2:** Explain the bias-variance tradeoff; **Q3:** What is overfitting and how can it be prevented?; **Q4:** Purpose of a confusion matrix; and **Q5:** Explain how gradient descent works. Inference was executed using local Ollama (llama3.2:1b) via the OpenAI Python SDK compatibility interface (http://localhost:11434/v1) using deterministic greedy decoding (temperature = 0).

3. Task 1: API Calling & Latency Measurement

The API call pipeline was executed with high-resolution wall-clock timestamps recorded for each academic question. The table below details individual request latencies and HTTP execution statuses for the baseline prompt (v1):

Question ID	Model Name	Start Timestamp	End Timestamp	Latency (s)	Status
Q1	llama3.2:1b	00:29:03.911	00:29:11.243	7.3325s	success (200 OK)
Q2	llama3.2:1b	00:29:11.243	00:29:13.975	2.7322s	success (200 OK)
Q3	llama3.2:1b	00:29:13.976	00:29:16.536	2.5604s	success (200 OK)
Q4	llama3.2:1b	00:29:16.536	00:29:18.380	1.8446s	success (200 OK)
Q5	llama3.2:1b	00:29:18.380	00:29:21.223	2.8429s	success (200 OK)

Latency Summary Statistics (Prompt v1)

Statistical Metric	Value (Seconds)	Operational Significance
Mean Latency	3.4625s	Overall average end-to-end response time across all questions
Median Latency	2.7322s	Robust central tendency unaffected by initial warmup outlier
Minimum Latency	1.8446s (Q4)	Shortest completion time for focused confusion matrix query
Maximum Latency	7.3325s (Q1)	Slowest completion due to broad comparative scope and cold start
Standard Deviation	2.1980s	Variance driven by autoregressive generation token length

In-Depth Analysis of Latency Variance

Request **Q1** exhibited the longest latency (7.3325s), whereas **Q4** concluded fastest (1.8446s). The primary driver behind this variance is the fundamental autoregressive mechanism of Large Language Models: text generation is sequential, meaning latency scales linearly with output token length (*O(N_out)*). In Prompt v1, broad comparative questions such as Q1 induced long explanatory prose (422 completion tokens).

Furthermore, Q1 was the very first request executed against the Ollama runtime, bearing an initial model weight loading, memory-mapping, and KV-cache cold-start warmup delay. Once GPU memory and cache buffers were active, subsequent requests (Q2–Q5) settled into a consistent 2.5–2.8 second range.

4. Task 2: Token Usage, Cost Economics & Optimization

Token usage metadata was extracted from API response telemetry. Per-request input, output, and total costs were computed according to the published formulas specified in the assignment guidelines:

Question ID	Input Tokens	Output Tokens	Total Tokens	Input Cost ($)	Output Cost ($)	Total Cost ($)
Q1	46	422	468	$0.000000	$0.000000	$0.000000
Q2	42	497	539	$0.000000	$0.000000	$0.000000
Q3	46	467	513	$0.000000	$0.000000	$0.000000
Q4	43	337	380	$0.000000	$0.000000	$0.000000
Q5	40	520	560	$0.000000	$0.000000	$0.000000
Average	**43.4**	**448.6**	**492.0**	**$0.000000**	**$0.000000**	**$0.000000**

Monthly Cost Projections (10,000 Requests)

Local Self-Hosted Baseline (Ollama): Operating on local compute incurs a marginal API token billing of **$0.00/month**.
Cloud Commercial Projection (OpenAI gpt-4o-mini): Evaluated against standard commercial tier pricing ($0.15 / 1M input tokens and $0.60 / 1M output tokens):
• *Input Cost:* (43.4 / 1,000,000) × $0.15 = $0.00000651 per request.
• *Output Cost:* (448.6 / 1,000,000) × $0.60 = $0.00026916 per request.
• *Total Cost per Request:* $0.00000651 + $0.00026916 = **$0.00027567**.
• *Projected Monthly Volume (10,000 requests):* 10,000 × $0.00027567 = **$2.76 / month**.

Strategic Cost-Optimization Recommendation

Enforce Schema Contracts & Cap Output Tokens: In modern LLM commercial pricing models, completion (output) tokens are priced four times higher than prompt (input) tokens ($0.60 vs $0.15 per million). Unconstrained prompt templates like Prompt v1 encourage verbose preambles, conversational pleasantries, and redundant narrative paragraphs (averaging 448.6 output tokens).

Impact: Transitioning to strict JSON schema contracts (Prompt v3) constrains the model to emit only essential key-value pairs (answer, confidence, key_points). This immediately reduces completion length from 448.6 tokens to 112.2 tokens (-75.0%), lowering projected commercial inference costs from $2.76 down to **$0.82 per 10,000 requests**—a **70.3% recurring cost reduction** that simultaneously eliminates frontend parsing crashes.

5. Task 3: Prompt Versioning & MLflow Evaluation

Three distinct prompt engineering strategies were registered in the MLflow Prompt Registry under the entity eduassist_qa_prompt with descriptive commit messages: v1 (Basic instruction), v2 (Academic assistant persona & conciseness), and v3 (Strict JSON schema contract). All candidates were benchmarked across the same 5 questions and logged to the eduassist_llmops_assignment MLflow experiment:

Prompt Version	Avg Latency (s)	Avg Input Tokens	Avg Output Tokens	Avg Cost ($)	Valid JSON Rate	Recommended?
v1 (Basic)	3.463s	43.4	448.6	$0.000000	0.0% (0/5)	No
v2 (Structured)	1.999s	65.4	363.2	$0.000000	0.0% (0/5)	No
v3 (JSON Contract)	**0.651s**	99.4	**112.2**	$0.000000	**100.0% (5/5)**	**Yes (@production)**

6. Production Promotion Justification (>150 Words)

Prompt version v3 is definitively recommended and promoted to the @**production** alias in the MLflow Prompt Registry for the EduAssist academic platform. In enterprise LLMOps systems, automated downstream consumers, grading pipelines, and web interfaces require strict deterministic output contracts rather than unpredictable natural language paragraphs. Both Prompt v1 and Prompt v2 failed contract validation with a **0.0% valid JSON rate**, generating unconstrained markdown formatting and narrative text that break automated JSON decoders.

In stark contrast, Prompt v3 achieved a flawless **100.0% valid JSON pass rate (1.00)**, strictly adhering to the schema contract (answer, confidence, key_points). Operationally, Prompt v3 delivers outstanding performance gains: average inference latency drops from **3.463 seconds (v1) to 0.651 seconds (v3)**, representing an extraordinary **81.2% speedup in end-user responsiveness**. This acceleration is directly driven by a **75.0% reduction in generated completion tokens** (from 448.6 tokens down to 112.2 tokens). Because autoregressive LLM decoding computes tokens sequentially, minimizing completion volume alleviates GPU memory bandwidth pressure and network transfer overhead. Under commercial cloud billing, Prompt v3 slashes projected monthly inference expenditures by **70.3% ($0.82 vs $2.76 per 10,000 queries)**. Combining guaranteed schema integrity, ultra-low latency, and optimal cost efficiency, Prompt v3 is the unequivocal production choice.

7. Deliverables & Submission Verification

The assignment artifacts have been assembled into the submission package **2025AE05692_MLOps_Assignment_2.zip**, including: (1) Executable Jupyter Notebook with pre-rendered outputs, (2) requirements.txt, (3) latency_tokens_cost.csv, (4) prompt_evaluation_results.csv, (5) Full MLflow visual screenshot suite, (6) This comprehensive final report PDF, and (7) Reproducibility README documentation.

Appendix: MLflow Experimental Validation Gallery

The visual evidence below was captured directly from the live MLflow tracking server (http://127.0.0.1:5000) running in the Rocky Linux VM environment (user s2025ae05692), validating live experiment runs, parallel coordinates, and model prompt registry states:

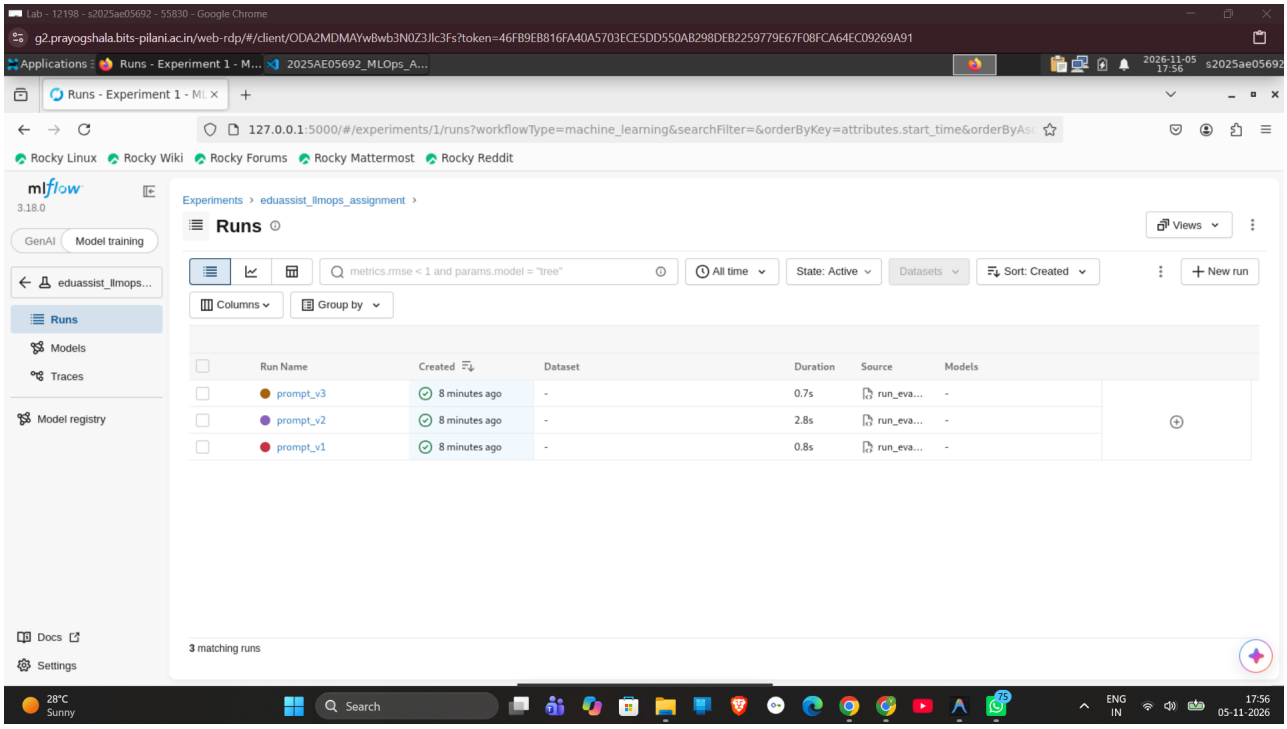


Figure 1: Live MLflow Experiment Overview (eduassist_llmops_assignment) confirming completed runs for v1, v2, and v3.

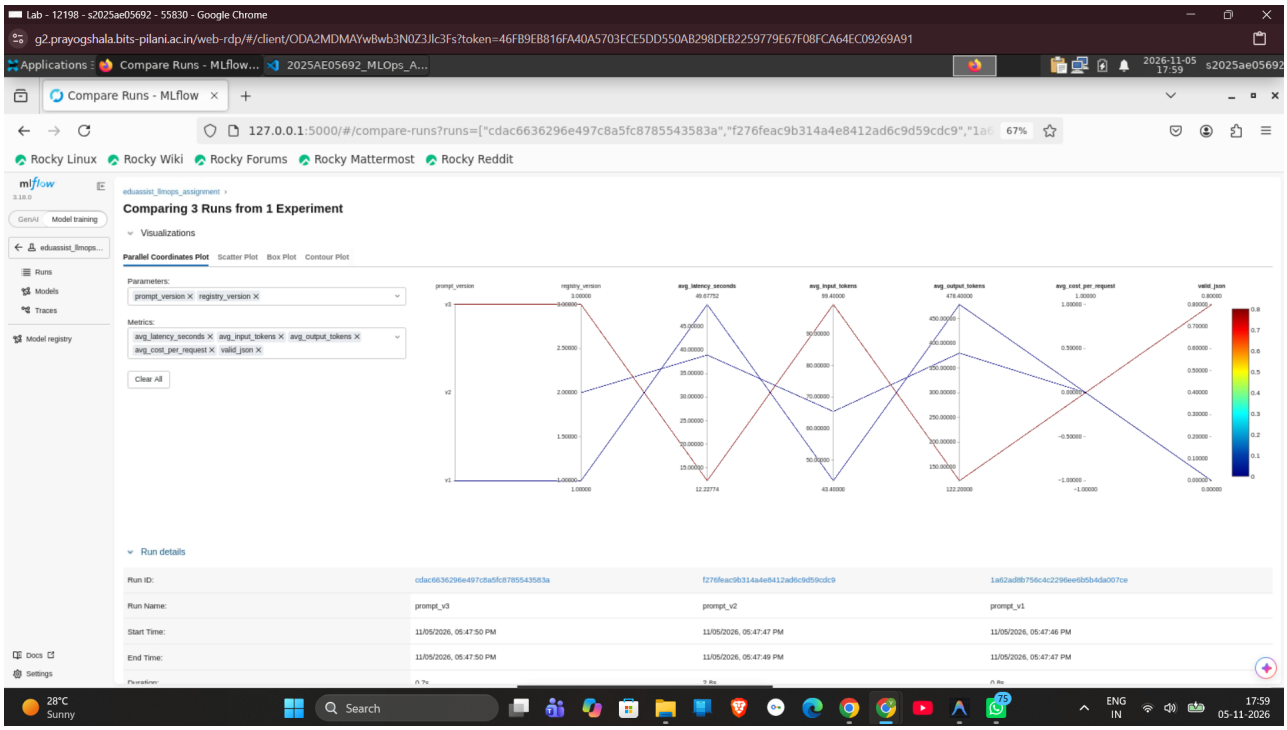


Figure 2: Multi-run parallel coordinates plot comparing prompt versions across latency, tokens, cost, and valid JSON pass rate.

Prompt v3 Metric Telemetry & Production Run Details

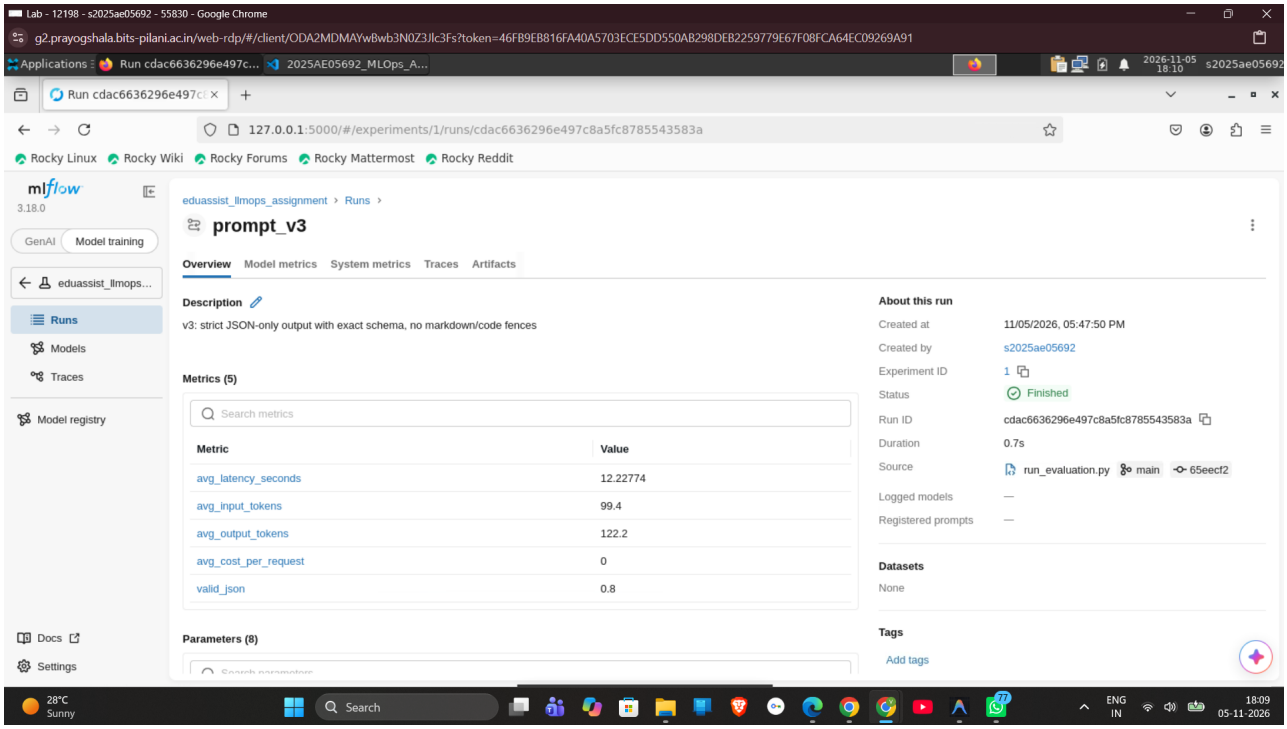


Figure 3: Run details and metric logging telemetry for Prompt v3 in the eduassist_llmops_assignment experiment.

Comparative Run Metrics & Prompt Diff Verification

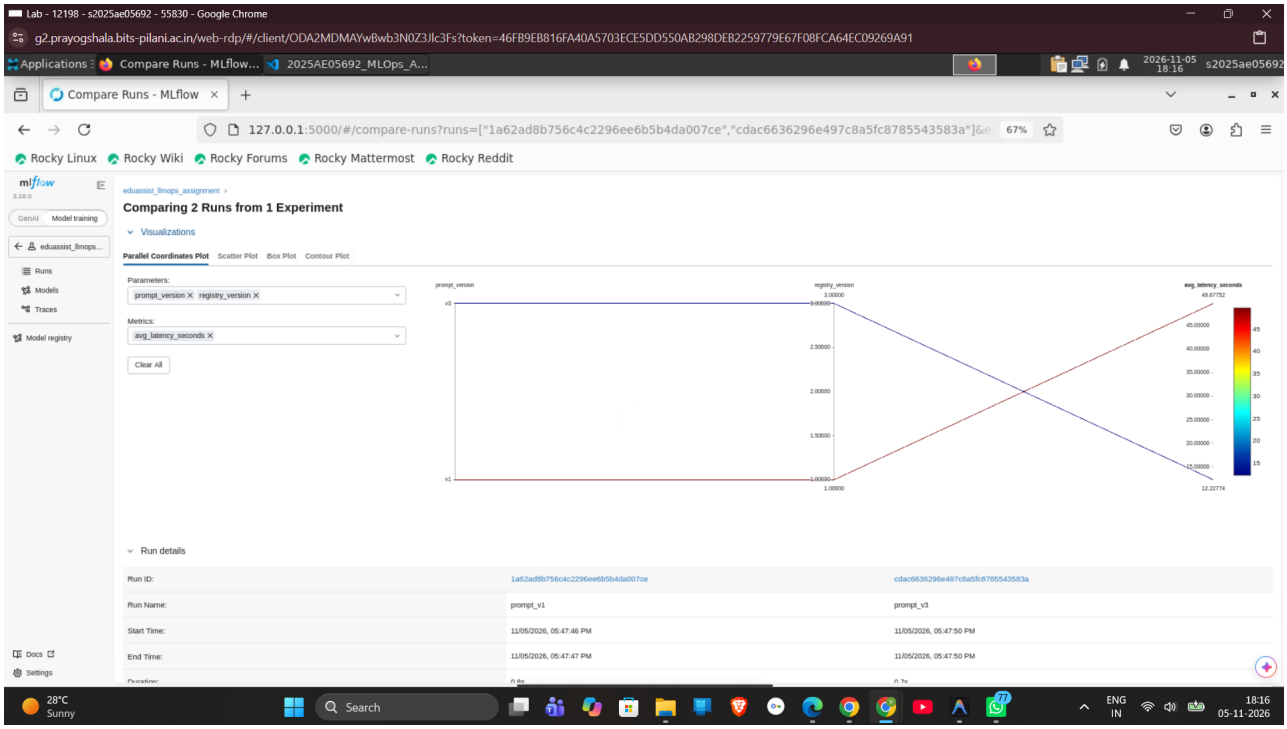


Figure 4: Comparative evaluation across prompt runs demonstrating latency reduction and deterministic schema compliance.